

CN: Data Structure and Algorithm

CC: BCSM2103

By

Dr. Rustam Ali Ahmed

The following topics belong to **Unit I** of your **Data Structure and Algorithms (FCCT2101)** syllabus. This unit lays the foundation for understanding all subsequent data structures.

UNIT I: Introduction to Data Structures

1. Basic Terminologies

Before studying data structures, it is important to understand some common terms.

Data

Data is a collection of raw facts, numbers, characters, or symbols that have no meaning by themselves.

Examples:

- 25
 - "Rustam"
 - 95
 - 'A'
-

Information

Information is processed or organized data that has meaning.

Example:

Data	Information
82, 75, 90	Student's marks
101, Amit, CSE	Student record

Data Item

A single unit of data.

Example:

Age = 20

Here **20** is a data item.

Group Item

A collection of related data items.

Example:

```
Student
-----
Roll No
Name
Department
CGPA
```

Entity

An object or person about which data is stored.

Examples:

- Student
 - Employee
 - Product
 - Book
-

Attribute

A property of an entity.

Example

Student

- Roll Number
- Name
- Age
- Department

These are attributes.

Record

A complete set of attributes describing one entity.

Example

Roll Name Department

101 Rahul CSE

This complete row is called a **record**.

File

A collection of records.

Example

Student File

101 Rahul CSE

102 Priya IT

103 Aman AI

Database

A collection of related files organized for efficient access.

Example

University Database

- Student File
 - Faculty File
 - Course File
 - Result File
-

2. Elementary Data Organization

Data organization refers to arranging data in a structured way for efficient storage and retrieval.

There are different levels of data organization:

Bit

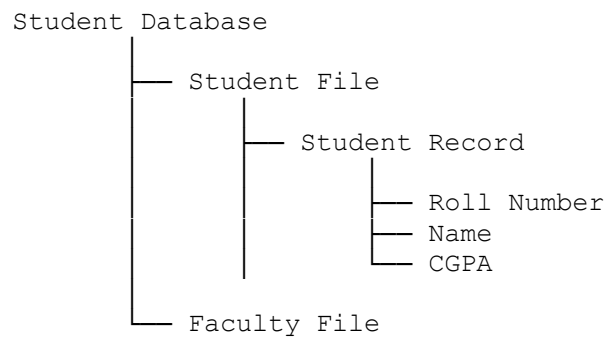
↓

Byte

↓

Field
↓
Record
↓
File
↓
Database

Example



Why is Data Organization Important?

- Easy searching
- Faster processing
- Better storage utilization
- Reduced redundancy
- Improved data security

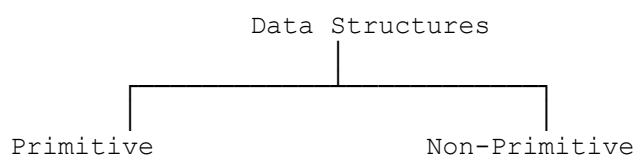
3. Data Structure

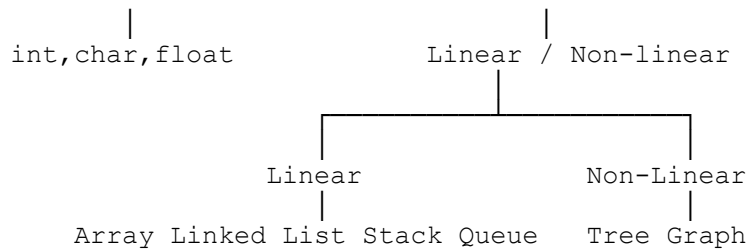
A Data Structure is a systematic way of organizing and storing data so that it can be accessed and modified efficiently.

Definition

A data structure is a logical or mathematical model used to organize data and the operations that can be performed on it.

Types of Data Structures





4. Data Structure Operations

Every data structure supports certain fundamental operations.

(a) Traversal

Traversal means visiting every element of a data structure exactly once.

Example

Array

10 20 30 40 50

Traversal

10 → 20 → 30 → 40 → 50

Time Complexity

O(n)

Applications

- Display data
 - Count elements
 - Find maximum/minimum
-

(b) Insertion

Insertion means adding a new element into the data structure.

Example

Before

10 20 40 50

Insert **30**

After

10 20 30 40 50

Time Complexity

- Beginning $\rightarrow O(n)$
 - End (Array) $\rightarrow O(1)$ (if space available)
 - Linked List $\rightarrow O(1)$ after locating the position
-

(c) Deletion

Removing an existing element.

Example

Before

10 20 30 40 50

Delete **30**

After

10 20 40 50

Time Complexity

Array $\rightarrow O(n)$

Linked List $\rightarrow O(1)$ after locating the node

(d) Searching

Finding whether an element exists.

Example

Search 40

10 20 30 40 50

Found

Common searching algorithms

- Linear Search
 - Binary Search
-

(e) Sorting

Arranging data in ascending or descending order.

Example

Before

40 10 30 20

After

10 20 30 40

(f) Merging

Combining two similar data structures.

Example

A = 10 20 30

B = 40 50

Merged

10 20 30 40 50

5. Analysis of an Algorithm

Algorithm analysis measures the efficiency of an algorithm in terms of **time** and **memory** required.

Instead of measuring execution time on a particular computer, we analyze how the algorithm grows as the input size (**n**) increases.

Example

Linear Search

```
for (i=0; i<n; i++)  
{  
    if (A[i]==key)  
        return i;  
}
```

Worst case:

Element is not found.

Comparisons = **n**

Time Complexity = **O(n)**

Why Analyze Algorithms?

- Compare different algorithms.
 - Select the most efficient solution.
 - Predict performance for large datasets.
 - Optimize resource utilization.
-

6. Asymptotic Notations

Asymptotic notations describe the growth rate of an algorithm as the input size becomes very large.

(a) Big O Notation — O()

Represents the **upper bound** (worst-case performance).

Example

Linear Search

$O(n)$

Merge Sort

$O(n \log n)$

(b) Omega Notation — $\Omega()$

Represents the **lower bound** (best-case performance).

Example

Linear Search

$\Omega(1)$

when the element is found in the first position.

(c) Theta Notation — $\Theta()$

Represents the **tight bound**, meaning the algorithm performs within the same order in both best and worst cases.

Example

Accessing an array element

$\Theta(1)$

Common Complexity Classes

Complexity	Name	Example
$O(1)$	Constant	Array access
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Bubble Sort
$O(2^n)$	Exponential	Recursive Fibonacci
$O(n!)$	Factorial	Traveling Salesman (Brute Force)

7. Time–Space Trade-off

A **time-space trade-off** is a design principle where improving execution speed often requires using more memory, while reducing memory usage may increase execution time.

Example 1: Hash Table vs. Linear Search

Method	Time	Space
Linear Search	$O(n)$	$O(1)$
Hash Table	$O(1)$ average	$O(n)$

The hash table is much faster but requires additional memory.

Example 2: Merge Sort vs. Insertion Sort

Algorithm	Time	Extra Space
Merge Sort	$O(n \log n)$	$O(n)$
Insertion Sort	$O(n^2)$	$O(1)$

Merge Sort achieves better performance by using additional memory.

Summary

After completing this unit, students should be able to:

- Define basic data structure terminology.
- Explain elementary data organization.
- Describe common data structure operations (insertion, deletion, traversal, searching, sorting, and merging).
- Analyze algorithms based on time and space complexity.
- Interpret asymptotic notations: **Big O**, **Big Ω** , and **Big Θ** .
- Explain the concept of the **time-space trade-off** and its role in algorithm design.

These concepts correspond to **Unit I** of the prescribed syllabus and provide the theoretical foundation for later units on stacks, queues, linked lists, trees, graphs, and advanced algorithms.